

LESSON LEARNED

Proyek SIGAP — Sistem Informasi Gangguan dan Aspirasi Publik

Mata Kuliah Praktikum Pemrograman Web Berbasis Framework

Dosen Pengampu:

Dina Budhi Utami S. Kom., M. T.



Oleh:

Muhammad Narendra Gema Akbar Raihan	230102091
Zakqi Ramadhan	230102129
M. SAKTI GABITA	230102131
Andreansyah Rahmattullah	240102015

**PROGRAM STUDI TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS MUHAMMADIYAH BANDUNG
2026**

1. Gambaran Proyek

SIGAP (Sistem Informasi Gangguan dan Aspirasi Publik) merupakan aplikasi pelaporan kerusakan infrastruktur publik berbasis arsitektur client-server yang dikembangkan sebagai proyek integrasi mata kuliah Pemrograman Perangkat Bergerak Client Server (PPBCS) dan Pemrograman Web Berbasis Framework (PWBF). Sistem terdiri atas dua komponen utama, yaitu dashboard web berbasis Laravel 13 yang digunakan oleh admin atau petugas, serta aplikasi mobile berbasis Flutter yang digunakan oleh masyarakat. Kedua platform saling berkomunikasi melalui REST API menggunakan autentikasi berbasis Laravel Sanctum.

2. Tantangan dan Kendala yang Dihadapi

2.1 Konfigurasi Laravel 13 yang Berbeda dari Versi Sebelumnya

Kendala

Pada tahap awal pengembangan, tim mengalami kesulitan karena seluruh endpoint API yang dibuat tidak muncul saat menjalankan `php artisan route:list`. Hal tersebut menyebabkan aplikasi Flutter tidak dapat terhubung ke backend Laravel sehingga proses integrasi client-server tertunda.

Penyebab

Laravel 13 tidak lagi secara otomatis memuat file `routes/api.php` sebagaimana pada versi sebelumnya. Selain itu, tim juga masih menggunakan beberapa referensi tutorial Laravel versi lama sehingga konfigurasi awal menjadi kurang sesuai.

Solusi

Tim mendaftarkan file `routes/api.php` secara eksplisit melalui fungsi `withRouting()` pada `bootstrap/app.php`. Setelah konfigurasi diperbaiki, seluruh endpoint API berhasil terdaftar dan dapat diakses oleh Flutter.

Pelajaran

Framework terus mengalami perkembangan sehingga dokumentasi resmi dan breaking changes perlu dipelajari sebelum memulai pengembangan agar tidak bergantung sepenuhnya pada tutorial versi lama.

2.2 Instalasi Laravel Breeze dan Sistem Autentikasi

Kendala

Setelah proses instalasi Laravel Breeze selesai, aplikasi menampilkan error `Route [profile.edit] not defined` ketika pengguna berhasil login sehingga dashboard tidak dapat diakses.

Penyebab

Proses `php artisan breeze:install blade` tidak menghasilkan scaffold secara lengkap. Beberapa file seperti `ProfileController` maupun route `profile` tidak berhasil dibuat.

Solusi

Tim menjalankan kembali proses instalasi menggunakan opsi `--force`, kemudian membersihkan cache route dan view serta melakukan verifikasi menggunakan `php`

artisan route:list.

Pelajaran

Setelah melakukan instalasi package yang menghasilkan scaffold secara otomatis, hasil instalasi perlu diverifikasi terlebih dahulu sebelum pengembangan dilanjutkan.

2.3 Migration Database dan Perubahan Struktur Data

Kendala

Pada tahap pembangunan database, migration gagal dijalankan karena urutan pembuatan tabel tidak memperhatikan ketergantungan foreign key. Selain itu, perubahan kebutuhan sistem berupa penambahan kolom urgency dan perubahan nilai status laporan menyebabkan kesulitan dalam memodifikasi kolom bertipe enum.

Penyebab

Tabel reports dibuat sebelum tabel report_categories, sementara perubahan enum tidak didukung secara langsung oleh Laravel Schema Builder.

Solusi

Tim memperbaiki urutan migration sehingga tabel induk dibuat terlebih dahulu. Untuk perubahan enum digunakan migration tambahan dengan DB::statement() sehingga struktur kolom dapat diubah tanpa menghapus data yang sudah ada.

Pelajaran

Perancangan struktur database harus dilakukan secara matang sejak awal, terutama apabila terdapat relasi antar tabel dan kemungkinan perubahan kebutuhan sistem.

2.4 Implementasi Role, Permission, dan Laravel Sanctum

Kendala

Pada implementasi autentikasi dan otorisasi, middleware role dari Spatie tidak dikenali sehingga proses pembatasan hak akses gagal dijalankan. Selain itu, autentikasi API menggunakan Laravel Sanctum juga sempat tidak berjalan karena konfigurasi route API belum dimuat.

Penyebab

Versi package Spatie yang digunakan memiliki namespace middleware berbeda dengan referensi yang digunakan. Selain itu, model User belum sepenuhnya dikonfigurasi menggunakan trait HasRoles dan HasApiTokens.

Solusi

Namespace middleware diverifikasi langsung melalui folder vendor, kemudian disesuaikan pada konfigurasi aplikasi. Model User diperbarui menggunakan trait yang sesuai, sedangkan seluruh role dan permission diuji menggunakan Laravel Tinker sebelum diintegrasikan ke controller.

Pelajaran

Konfigurasi autentikasi dan otorisasi perlu diuji secara bertahap. Verifikasi

langsung terhadap struktur package lebih akurat dibandingkan hanya mengikuti tutorial.

2.5 Pengembangan REST API dan Integrasi Flutter

Kendala

Pada tahap integrasi, Flutter tidak dapat mengakses server Laravel menggunakan alamat localhost maupun 127.0.0.1. Selain itu, beberapa endpoint API memberikan respons yang tidak sesuai akibat konfigurasi CORS serta perbedaan mekanisme autentikasi antara web dan mobile.

Penyebab

Android Emulator memiliki jaringan virtual sendiri sehingga localhost mengarah ke emulator, bukan komputer host.

Solusi

Tim menggunakan alamat 10.0.2.2 sebagai base URL khusus Android Emulator, mengonfigurasi CORS Laravel, serta melakukan pengujian endpoint menggunakan Postman dan Thunder Client sebelum diintegrasikan ke Flutter.

Pelajaran

Pemahaman mengenai arsitektur jaringan emulator dan komunikasi REST API sangat penting dalam pengembangan aplikasi mobile berbasis client-server.

2.6 Penyimpanan Token, Upload Gambar, dan Manajemen State Flutter

Kendala

Sesi login pengguna tidak tetap tersimpan setelah aplikasi ditutup. Selain itu, proses upload foto laporan menggunakan Flutter mengalami kegagalan, dan tampilan aplikasi tidak otomatis diperbarui setelah operasi CRUD berhasil dilakukan.

Penyebab

Token autentikasi belum disimpan secara persisten, upload file memerlukan mekanisme multipart request, sedangkan manajemen state antar halaman belum dirancang dengan baik.

Solusi

Token disimpan menggunakan Shared Preferences, upload gambar dilakukan menggunakan MultipartRequest, dan Laravel dikonfigurasi menggunakan php artisan storage:link. Untuk sinkronisasi data antarscreen digunakan pola await Navigator.push() yang mengembalikan nilai boolean sehingga halaman sebelumnya dapat melakukan refresh data secara otomatis.

Pelajaran

Integrasi Flutter tidak hanya bergantung pada API, tetapi juga pada manajemen state, penyimpanan lokal, serta mekanisme komunikasi antar halaman.

2.7 Sinkronisasi Frontend-Backend dan Debugging

Kendala

Beberapa halaman Blade menampilkan error Undefined Variable, sementara proses debugging membutuhkan waktu cukup lama karena pesan error yang muncul kurang informatif.

Penyebab

Sebagian route masih menggunakan closure sehingga data dari controller belum dikirim dengan benar. Selain itu, debugging dilakukan pada banyak layer sekaligus, mulai dari database, controller, API, hingga Flutter.

Solusi

Route diubah menjadi controller yang terstruktur, variabel dikirim menggunakan compact(), serta helper Auth::user() dimanfaatkan ketika diperlukan. Proses debugging dilakukan secara sistematis melalui laravel.log, php artisan route:list, dan pengujian bertahap pada setiap layer sistem.

Pelajaran

Debugging pada aplikasi client-server harus dilakukan secara sistematis dengan mengidentifikasi sumber masalah mulai dari database, backend, API, hingga frontend.

2.8 Kolaborasi Tim dan Pengelolaan GitHub

Kendala

Pada tahap awal proyek, kontribusi GitHub belum merata karena hanya sebagian anggota yang menangani setup awal. Selain itu, beberapa kali terjadi merge conflict akibat perubahan pada file yang sama.

Penyebab

Pembagian tugas belum cukup rinci dan belum diterapkan strategi branching yang jelas.

Solusi

Tim menerapkan feature branch, pull request, dan code review. Pembagian tugas juga disusun ulang sehingga setiap anggota memperoleh tanggung jawab pada pengembangan Laravel maupun Flutter dan dapat bekerja secara paralel.

Pelajaran

Kolaborasi yang baik memerlukan pembagian tugas yang jelas, komunikasi yang intensif, serta penggunaan Git sebagai alat koordinasi, bukan sekadar tempat penyimpanan kode.

3. Hal-Hal yang Dipelajari dari Proyek Ini

3.1 Memahami Arsitektur Client-Server dan MVC Full-Stack

Melalui proyek ini tim memperoleh pemahaman yang lebih mendalam mengenai penerapan arsitektur client-server menggunakan Laravel sebagai backend dan Flutter sebagai frontend. Konsep MVC pada Laravel dipadukan dengan pola Service Layer pada Flutter sehingga masing-masing komponen memiliki tanggung jawab yang jelas.

3.2 Pentingnya Perancangan Kontrak API

Tim menyadari bahwa dokumentasi endpoint, format request, dan response sebelum proses coding memungkinkan backend dan frontend dikembangkan secara paralel tanpa saling menunggu.

3.3 Git sebagai Media Kolaborasi

Penggunaan feature branch, pull request, dan code review membantu meminimalkan konflik kode serta memudahkan pelacakan kontribusi setiap anggota selama pengembangan.

3.4 Pentingnya Pengujian Bertahap

Pengujian menggunakan Postman, Thunder Client, serta pengujian langsung pada aplikasi Flutter membantu menemukan kesalahan lebih awal sebelum memengaruhi modul lain yang telah terintegrasi.

3.5 Pentingnya Komunikasi Tim

Sebagian besar kendala selama pengembangan bukan berasal dari teknologi yang digunakan, melainkan dari kurangnya komunikasi antaranggota mengenai pembagian tugas, relasi model, maupun perubahan kebutuhan sistem.

3.6 Manajemen Scope dan Waktu

Dengan waktu pengembangan yang relatif singkat dan beberapa checkpoint, tim belajar memprioritaskan fitur utama terlebih dahulu sebelum mengembangkan fitur tambahan sehingga seluruh kebutuhan proyek dapat diselesaikan tepat waktu.

4. Kesimpulan

Proyek SIGAP berhasil mengintegrasikan aplikasi web berbasis Laravel 13 dengan aplikasi mobile berbasis Flutter melalui REST API dalam satu sistem client-server yang fungsional. Berbagai kendala teknis, mulai dari perubahan konfigurasi Laravel 13, implementasi autentikasi dan otorisasi, migration database, integrasi Flutter, manajemen state, hingga kolaborasi tim melalui GitHub berhasil diselesaikan melalui proses analisis, dokumentasi, dan pengujian yang sistematis.